

RESEARCH PUBLICATION MANAGEMENT SYSTEM

1. Introduction

Research activities in universities and academic institutions generate a large amount of information related to researchers, departments, publications, conferences, journals, and research collaborations. Managing such information manually can be time-consuming and may lead to inconsistencies in data maintenance. To address this challenge, a centralized and efficient system is required for storing, organizing, and managing research-related records.

The Research Publication Management System is a backend-based application developed using Spring Boot and MongoDB. The system provides a structured platform for managing research entities and their relationships through RESTful APIs. It enables administrators and researchers to perform various operations such as adding new records, updating existing information, retrieving data, and removing outdated entries.

The project focuses on implementing modern backend development concepts including REST API development, database integration, validation handling, exception management, and relationship mapping using MongoDB. API documentation and testing are facilitated through Swagger OpenAPI, allowing users to interact with the system without the need for a frontend interface.

The system serves as a practical example of how backend technologies can be used to develop scalable and maintainable applications for academic and research environments.

2. Objectives

The primary objectives of the project are:

- To develop a centralized system for managing research-related information.
- To implement complete CRUD (Create, Read, Update, Delete) operations for all major entities.
- To establish and manage relationships between researchers, departments, publications, journals, and conferences.
- To provide RESTful APIs for efficient data management.
- To integrate MongoDB as a NoSQL database for storing and retrieving information.
- To demonstrate backend application development using Spring Boot.
- To provide API documentation and testing support through Swagger OpenAPI.
- To create a modular and scalable architecture that can be extended in future versions.

3. Technology Stack

The following technologies were used during the development of the project:

Component	Technology
Programming Language	Java
Backend Framework	Spring Boot
Database	MongoDB
Data Access Layer	Spring Data MongoDB
API Testing	Postman
API Documentation	Swagger OpenAPI
Build Tool	Maven
IDE	IntelliJ IDEA

Spring Boot

Spring Boot simplifies the development of Java-based enterprise applications by providing embedded server support, dependency management, and rapid application development features.

MongoDB

MongoDB is a document-oriented NoSQL database used to store application data in flexible JSON-like documents. It offers high performance, scalability, and ease of integration with Spring Boot.

Swagger OpenAPI

Swagger provides an interactive user interface for testing and documenting REST APIs. It allows developers to verify endpoints without using external tools.

4. System Architecture

The application follows a layered architecture consisting of the following components:

Controller Layer

The controller layer exposes REST endpoints and handles incoming HTTP requests from clients.

Examples:

- DepartmentController
- ResearcherController
- PublicationController
- ConferenceController
- JournalController
- ResearchProfileController

Service Layer

The service layer contains business logic and coordinates communication between controllers and repositories.

Examples:

- DepartmentService
- ResearcherService
- PublicationService
- ConferenceService
- JournalService

Repository Layer

The repository layer interacts directly with MongoDB using Spring Data MongoDB repositories.

Examples:

- DepartmentRepository
- ResearcherRepository
- PublicationRepository
- ConferenceRepository

Database Layer

MongoDB stores all research-related records and relationship mappings.

5. System Modules

5.1 Department Management

This module manages department-related information.

Functions:

- Add new departments
- View department details
- Update department information
- Delete

departments Attributes:

- Department ID
- Department Name
- Department Head

5.2 Researcher Management

This module stores information about researchers associated with departments.

Functions:

- Create researcher records
- Retrieve researcher details
- Update researcher information
- Delete researcher

records Attributes:

- Researcher ID
- Researcher Name
- Email
- Designation
- Department ID

5.3 Publication Management

This module manages research publications.

Functions:

- Add publications
- Retrieve publication details
- Update publication information
- Delete

publications Attributes:

- Publication ID
- Title
- Abstract
- Publication Year

5.4 Conference Management

The conference module maintains information related to academic

conferences. Functions:

- Add conferences
- View conference details
- Update conference information
- Delete conference

records Attributes:

- Conference Name
 - Location
 - Year
-

5.5 Journal Management

The journal module stores information about journals used for publication.

Functions:

- Create journal records
- Retrieve journal details
- Update journal information
- Delete journal

records Attributes:

- Journal Name
 - Publisher
 - Impact Factor
-

5.6 Research Profile Management

The research profile module maintains detailed information about researchers.

Attributes:

- Researcher ID
 - Specialization
 - Experience Years
-

6. Database Design and Relationships

The system implements several database relationships to represent real-world academic structures.

One-to-One Relationship

Researcher ↔ Research Profile

Each researcher is associated with exactly one research profile, and each profile belongs to a single researcher.

One-to-Many Relationship

Department → Researchers

A department can have multiple researchers working under it.

Many-to-One Relationship

Researchers → Department

Multiple researchers can belong to the same department.

Many-to-Many Relationship

Researchers ↔ Publications

A researcher can contribute to multiple publications, and a publication can have multiple authors. This relationship is implemented through the Publication Author collection.

7. CRUD Operations

The application supports complete CRUD functionality for all major entities.

Create Operation

Used to add new records into the database. Examples:

- Create Department
- Create Researcher
- Create Publication
- Create Conference

Read Operation

Used to retrieve existing records.

Examples:

- View all departments
- View all researchers
- Search publications
- Search researchers

Update Operation

Used to modify existing information.

Examples:

- Update department details
- Update researcher information
- Update conference details

Delete Operation

Used to remove records from the database.

Examples:

- Delete departments
- Delete publications
- Delete conferences

8. API Testing and Documentation

The application APIs were tested using Swagger OpenAPI.

Swagger provides:

- Interactive API testing
- Request and response visualization
- API documentation
- Validation of endpoints

The Swagger interface allows users to execute API requests directly through a web browser.

9. Additional Features

Scholar Dashboard

A dashboard module was implemented to provide quick insights into the system.

The dashboard displays:

- Current system status
- Current date and time
- Motivational research message
- Total departments
- Total researchers
- Total publications
- Total journals
- Total conferences

Publication Search

Users can search publications using keywords and retrieve matching results.

Researcher Search

Researchers can be searched based on their names for quick access to information.

Validation and Exception Handling

The system includes validation mechanisms to ensure data integrity and prevent invalid inputs. Global exception handling is implemented to provide meaningful error responses.

10. Results and Discussion

The application was successfully developed and integrated with MongoDB. All modules were tested through Swagger and MongoDB Compass. CRUD operations functioned correctly for every entity. Relationship mappings were successfully implemented and verified through database records.

The system demonstrates how Spring Boot and MongoDB can be used together to build scalable backend applications for academic and research environments.

11. Future Enhancements

The project can be extended with the following features:

- User Authentication and Authorization
- Role-Based Access Control
- AI-Based Research Recommendations
- Automatic Keyword Extraction
- Research Trend Analysis
- Email Notifications

- File Upload for Research Papers
- Collaboration Suggestions
- Frontend Development using React or Angular
- Cloud Deployment

Outcome

The Research Publication Management System was successfully developed and implemented using Spring Boot and MongoDB. The system provides a centralized platform for managing departments, researchers, publications, conferences, journals, and research profiles through RESTful APIs. Complete CRUD functionality was achieved for all major entities, enabling efficient creation, retrieval, modification, and deletion of records. The project successfully demonstrates MongoDB integration, API development, input validation, exception handling, and relationship mapping, including One-to-One, One-to-Many, Many-to-One, and Many-to-Many associations. Additional features such as the Scholar Dashboard, publication search, and researcher search enhance the usability of the system. The APIs were tested using Swagger, and all functionalities performed as expected. The project meets the intended objectives and serves as a practical backend solution for research publication management in academic and research institutions.

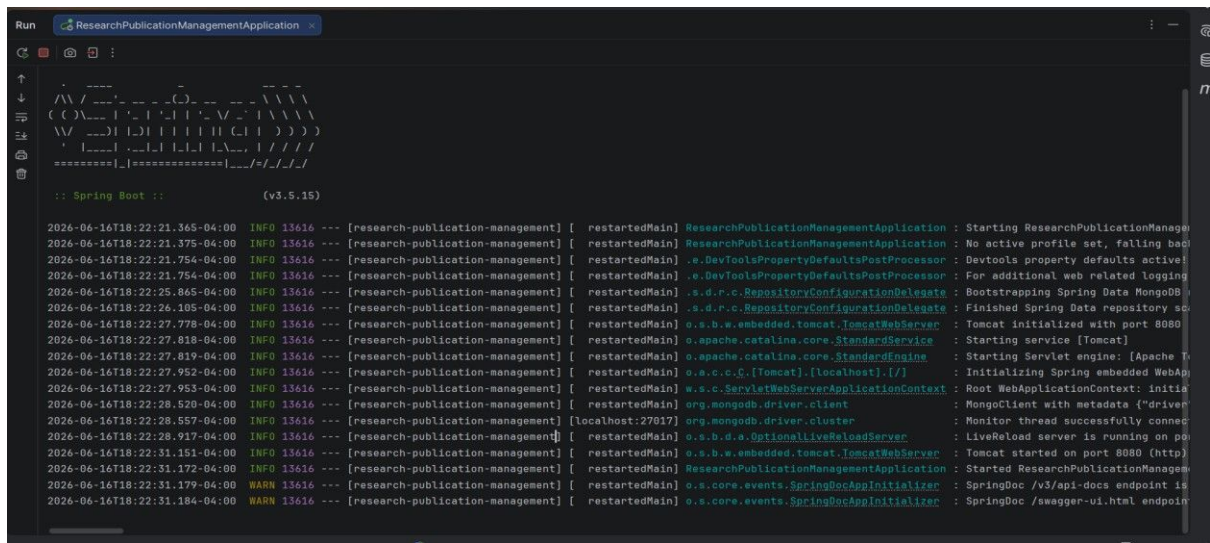
12. Conclusion

The Research Publication Management System successfully demonstrates the development of a backend application using Spring Boot and MongoDB. The system provides a structured platform for managing departments, researchers, publications, conferences, journals, and research profiles through REST APIs. Complete CRUD functionality, validation handling, exception management, and relationship mapping have been implemented successfully.

The project fulfills the academic requirements of backend development while also representing a practical real-world application for research data management. Its modular architecture allows future enhancements and makes it a strong foundation for building a comprehensive research management platform.

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure 0: Successful startup of Spring Boot application and MongoDB database connection.



```
Run ResearchPublicationManagementApplication x
...
:: Spring Boot :: (v3.5.15)

2026-06-16T18:22:21.365-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] ResearchPublicationManagementApplication : Starting ResearchPublicationManagementApplication
2026-06-16T18:22:21.375-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] ResearchPublicationManagementApplication : No active profile set, falling back to default profiles
2026-06-16T18:22:21.754-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.devtools.properties.DevToolsPropertyDefaultsPostProcessor : DevTools property defaults active
2026-06-16T18:22:21.754-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.devtools.properties.DevToolsPropertyDefaultsPostProcessor : For additional web related logging
2026-06-16T18:22:25.865-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data MongoDB
2026-06-16T18:22:26.105-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning
2026-06-16T18:22:27.778-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080
2026-06-16T18:22:27.819-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-16T18:22:27.819-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.90]
2026-06-16T18:22:27.952-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.c.g.Cat[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-16T18:22:27.953-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed
2026-06-16T18:22:28.520-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] org.mongodb.driver.client : MongoClient with metadata {"driver": "SpringDataMongoDB", "platform": "Spring Boot", "version": "3.5.15"}
2026-06-16T18:22:28.520-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] org.mongodb.driver.cluster : Monitor thread successfully connected to server
2026-06-16T18:22:28.520-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] org.mongodb.driver.cluster : LiveReload server is running on port 27017
2026-06-16T18:22:31.151-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http)
2026-06-16T18:22:31.172-04:00 INFO 15616 --- [research-publication-management] [ restartedMain] ResearchPublicationManagementApplication : Started ResearchPublicationManagementApplication
2026-06-16T18:22:31.179-04:00 WARN 15616 --- [research-publication-management] [ restartedMain] o.s.core.events.SpringDocAppInitializer : SpringDoc /v3/api-docs endpoint is disabled
2026-06-16T18:22:31.184-04:00 WARN 15616 --- [research-publication-management] [ restartedMain] o.s.core.events.SpringDocAppInitializer : SpringDoc /swagger-ui.html endpoint is disabled
```

Figure 1: Spring Boot project structure implementing layered architecture.

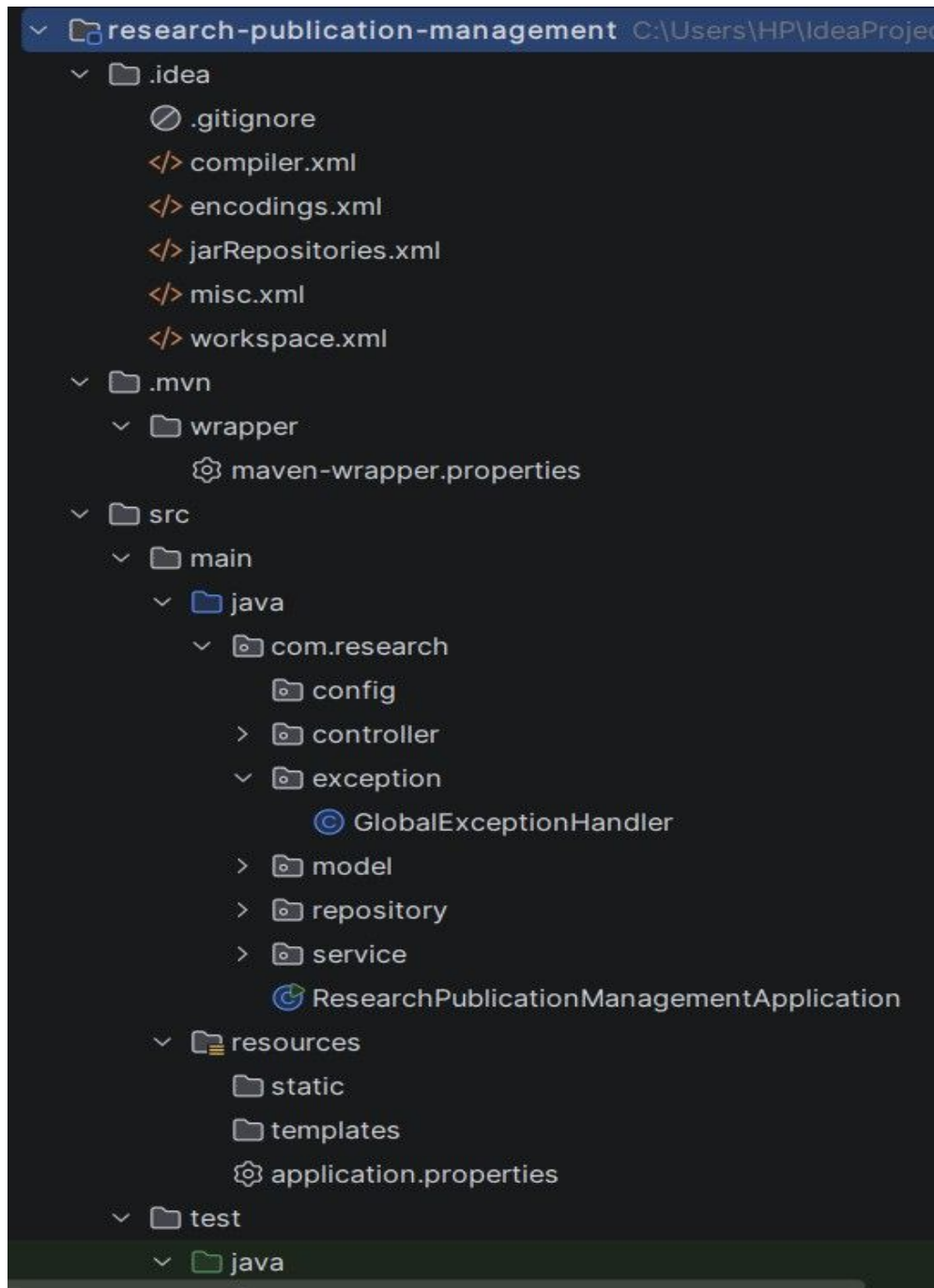
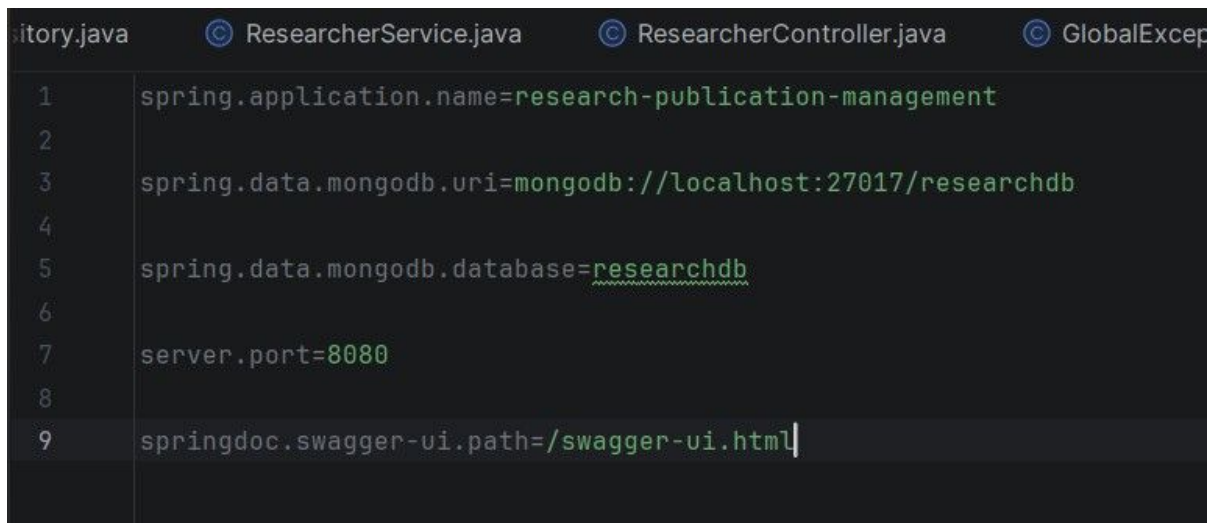
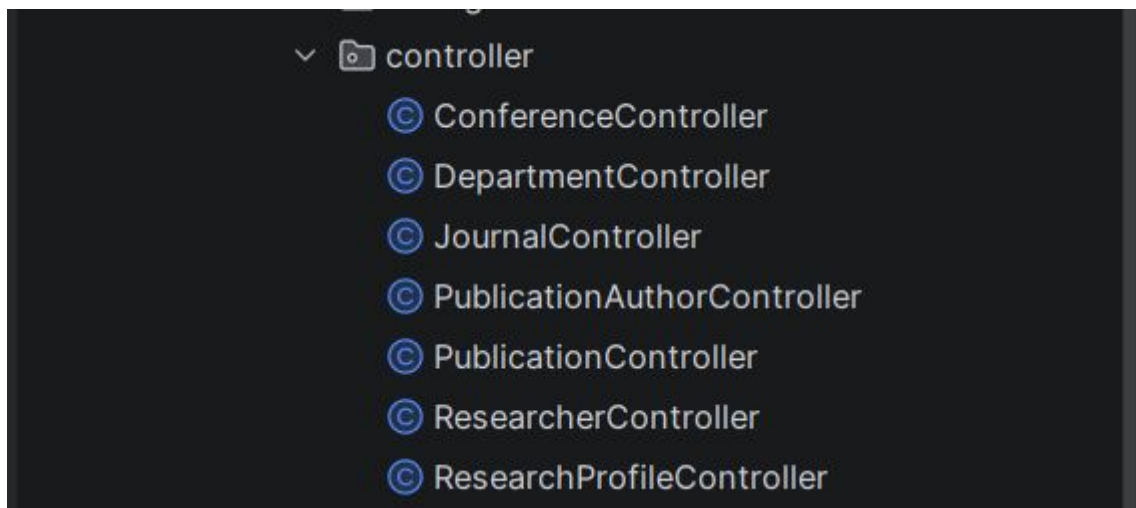


Figure 2: MongoDB database configuration.



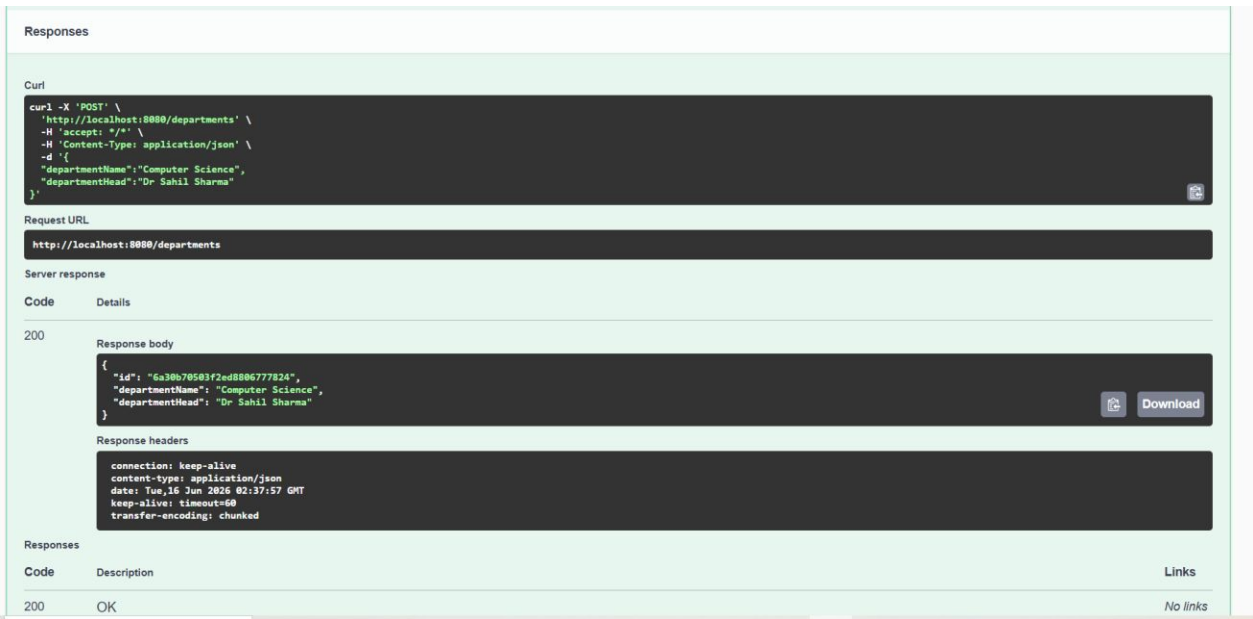
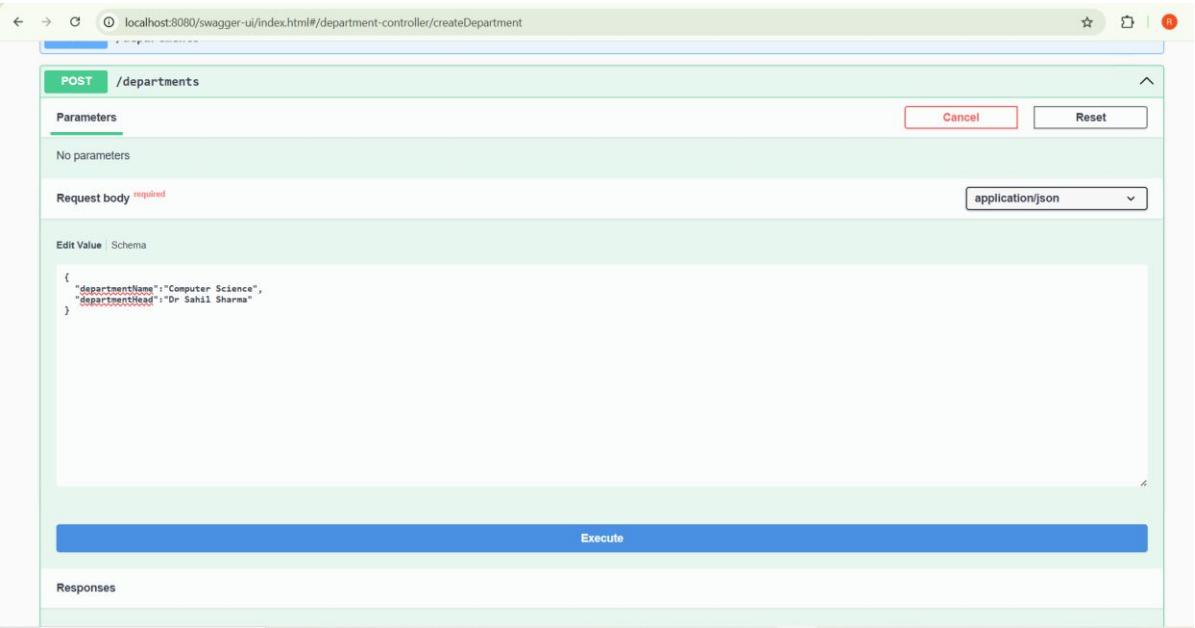
```
itory.java    ResearcherService.java    ResearcherController.java    GlobalExcep
1    spring.application.name=research-publication-management
2
3    spring.data.mongodb.uri=mongodb://localhost:27017/researchdb
4
5    spring.data.mongodb.database=researchdb
6
7    server.port=8080
8
9    springdoc.swagger-ui.path=/swagger-ui.html
```

Figure 3: REST APIs exposed through Swagger UI.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure 4: Department creation using REST API.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure 5: Input validation using Spring Validation.

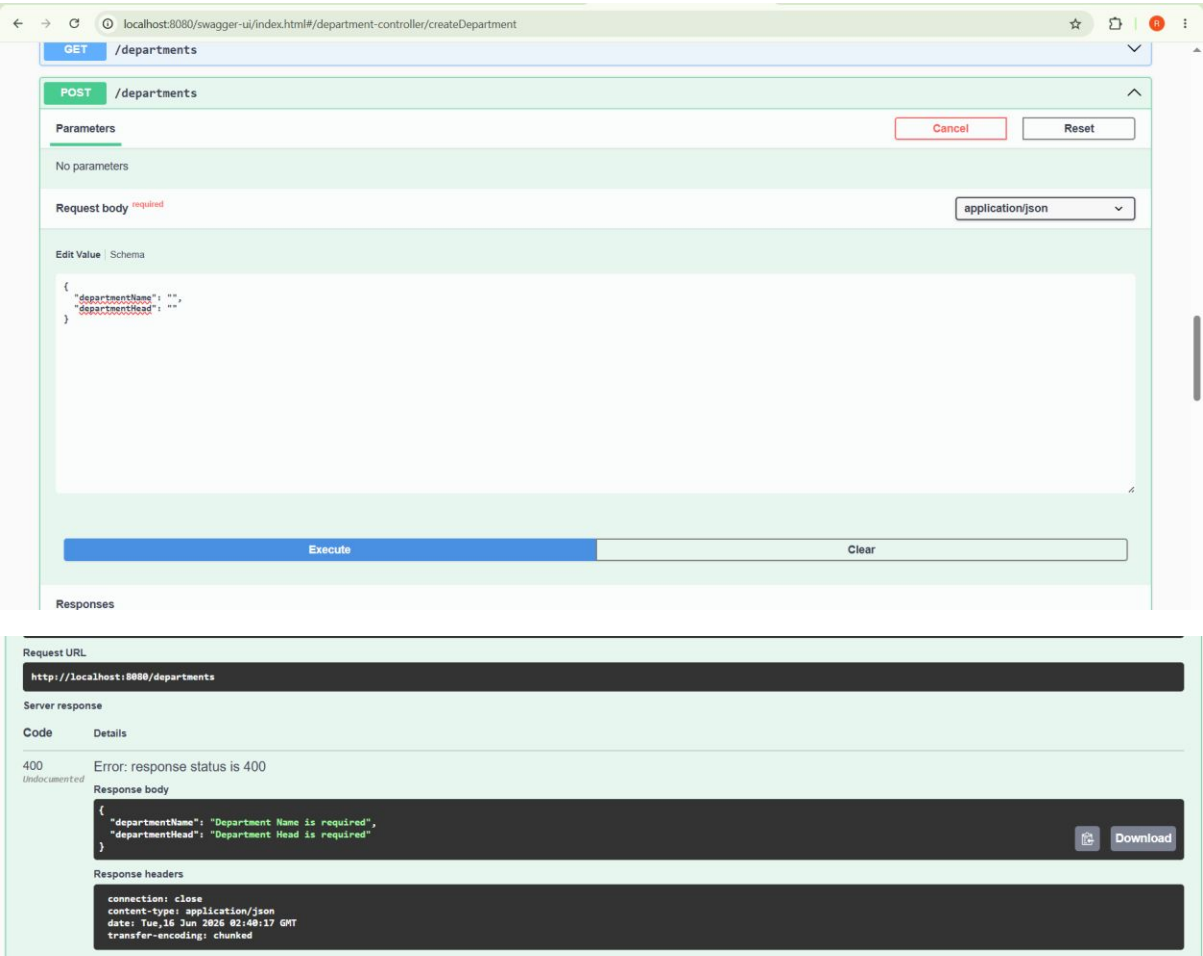
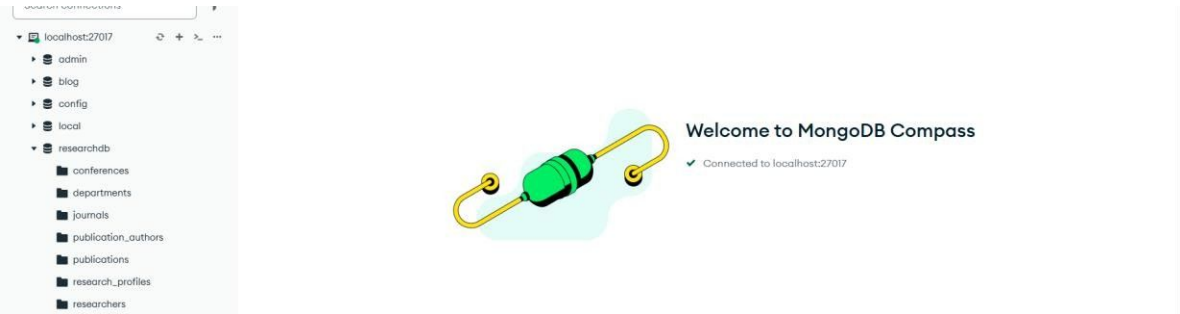
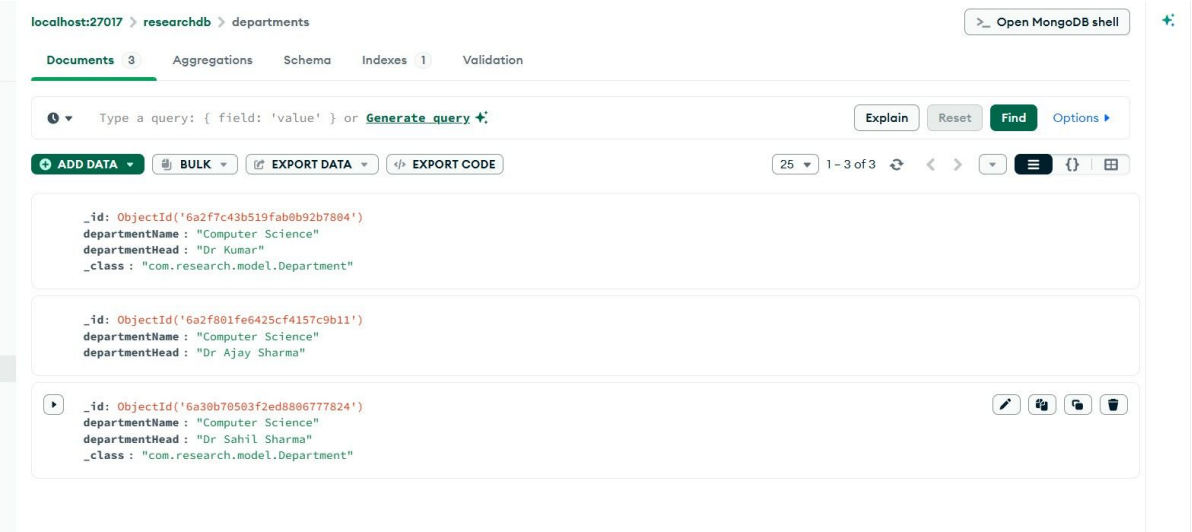


Figure 6: MongoDB collections created by the application.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

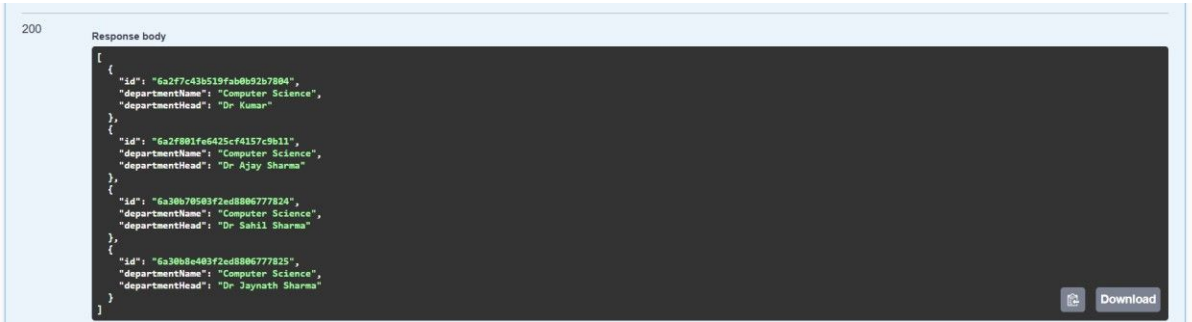
Figure 7: Department records stored in MongoDB.



CRUD
OPERATION:
CREATE:



READ:



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

UPDATE:

200

Response body

```
{
  "id": "6a2f7c43b519fab8b92b7884",
  "departmentName": "Computer Science",
  "departmentHead": "Dr Sachin Kumar"
}
```

Download

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue, 16 Jun 2026 02:49:05 GMT
keep-alive: timeout=60
transfer-encoding: chunked
```

Responses

DELETE:

200

Response body

Department Deleted Successfully

Download

Response headers

```
connection: keep-alive
content-length: 31
content-type: text/plain; charset=UTF-8
date: Tue, 16 Jun 2026 02:50:30 GMT
keep-alive: timeout=60
```

Responses

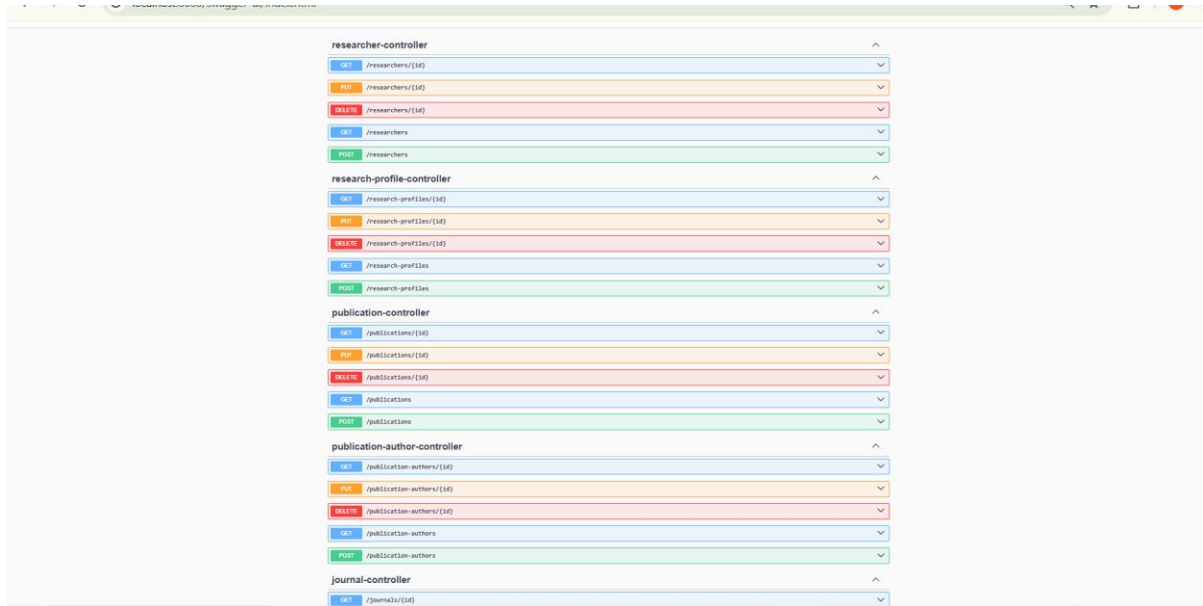
Code	Description	Links
------	-------------	-------

MONGODB DEPARTMENT

<pre>_id: ObjectId('6a2f7c43b519fab8b92b7884') departmentName: "Computer Science" departmentHead: "Dr Kumar" _class: "com.research.model.Department"</pre>
<pre>_id: ObjectId('6a2f801fe6425cf4157c9b11') departmentName: "Computer Science" departmentHead: "Dr Ajay Sharma"</pre>
<pre>_id: ObjectId('6a30b70503f2ed8806777824') departmentName: "Computer Science" departmentHead: "Dr Sahil Sharma" _class: "com.research.model.Department"</pre>

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

SCHEMA :



This screenshot displays the Swagger UI for the AI Assisted Research Publication Management System. It lists the following API endpoints:

Controller	Method	Endpoint
researcher-controller	GET	/researchers/{id}
	POST	/researchers/{id}
	DELETE	/researchers/{id}
	GET	/researchers
	POST	/researchers
research-profile-controller	GET	/research-profiles/{id}
	POST	/research-profiles/{id}
	DELETE	/research-profiles/{id}
	GET	/research-profiles
	POST	/research-profiles
publication-controller	GET	/publications/{id}
	POST	/publications/{id}
	DELETE	/publications/{id}
	GET	/publications
	POST	/publications
publication-author-controller	GET	/publication-authors/{id}
	POST	/publication-authors/{id}
	DELETE	/publication-authors/{id}
	GET	/publication-authors
	POST	/publication-authors
journal-controller	GET	/journals/{id}



This screenshot displays the Swagger UI for the AI Assisted Research Publication Management System, showing additional API endpoints:

Controller	Method	Endpoint
journal-controller	GET	/journals/{id}
	POST	/journals/{id}
	DELETE	/journals/{id}
	GET	/journals
	POST	/journals
department-controller	GET	/departments/{id}
	POST	/departments/{id}
	DELETE	/departments/{id}
	GET	/departments
	POST	/departments
conference-controller	GET	/conferences/{id}
	POST	/conferences/{id}
	DELETE	/conferences/{id}
	GET	/conferences
	POST	/conferences

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

FINAL SCHEMA:



Fig: Scholar Dashboard displaying real-time statistics, system status, date, time, and motivational research insights.

- Publication search functionality enabling keyword-based retrieval of research publications.
- Researcher search functionality for locating researchers based on their names.
- Researcher search functionality for locating researchers based on their names.

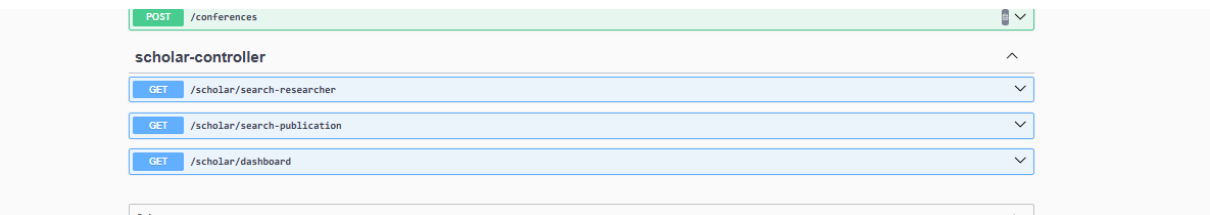
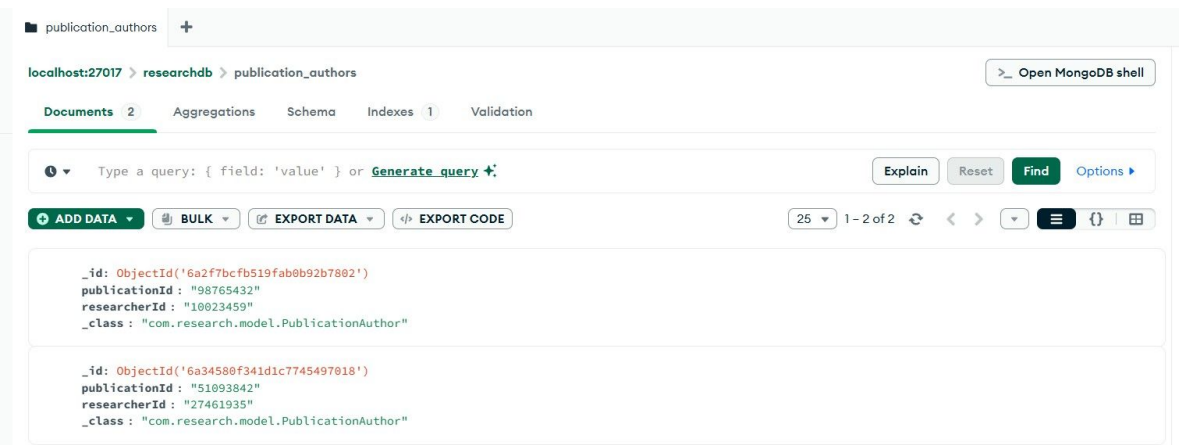
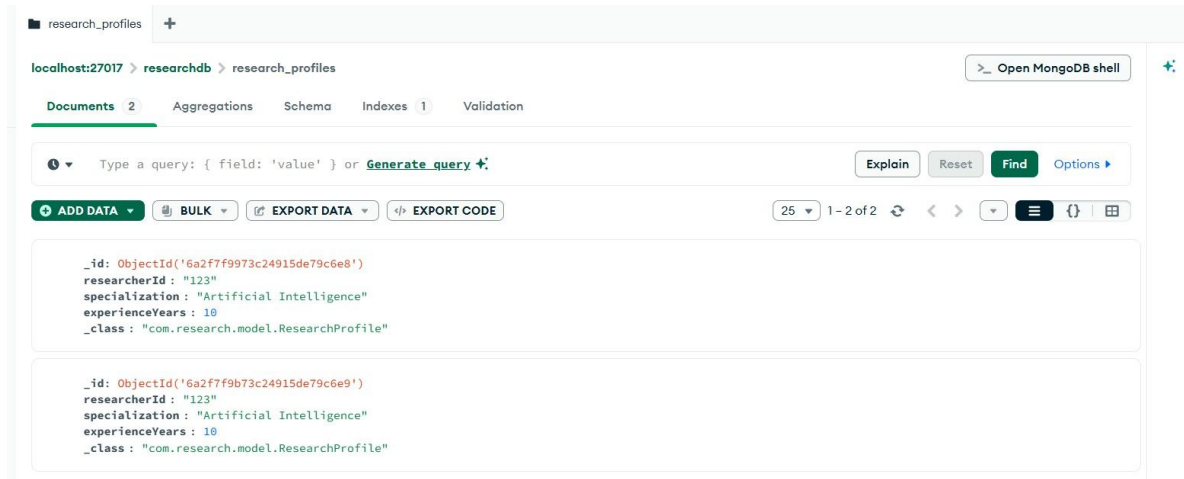


Figure : Many-to-Many relationship mapping between Researchers and Publications using the Publication Author collection.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure : One-to-One relationship implementation between Researcher and Research Profile entities.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure : Scholar Dashboard displaying real-time research statistics, system status, date, and time.

GET

/scholar/dashboard

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/scholar/dashboard' \
  -H 'accept: */*'

```

Request URL

http://localhost:8080/scholar/dashboard

Server response

Code

Details

200

Response body

```
{
  "currentDate": "2025-06-16",
  "currentTime": "18:45:46",
  "motivation": "The best way to predict the future is to invent it.",
  "totalDepartments": 0,
  "totalResearchers": 0,
  "totalPublications": 0,
  "totalJournals": 0,
  "totalConferences": 0,
  "status": "ACTIVE"
}
```

Response headers

```
connection: keep-alive
content-type: application/json
date: Tue, 16 Jun 2026 22:45:46 GMT
keep-alive: timeout=60
transfer-encoding: chunked

```

Responses

Code	Description	Links
200	OK	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "currentDate": "string",
  "currentTime": "string",
  "motivation": "string",
  "totalDepartments": 0,
  "totalResearchers": 0,
  "totalPublications": 0,
  "totalJournals": 0,
  "totalConferences": 0,
  "status": "string"
}
```

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure : Publication search functionality enabling keyword-based retrieval of research publications.

GET /scholar/search-publication

Parameters

Cancel

Name	Description
keyword * required	
string	AI
(query)	

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8080/scholar/search-publication?keyword=AI' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8080/scholar/search-publication?keyword=AI
```

Server response

Code	Details			
200	Response body <pre>{ "id": "6a2f7baab519fab8b52b7801", "title": "AI Assisted Research Publication Management", "publicationYear": 2020, "conferenceId": "someConferenceId" }, { "id": "6a38c278b50214078d0c2025", "title": "Cyber Security using AI", "publicationYear": 2020, "conferenceId": "CONF003" } }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Tue, 16 Jun 2020 22:48:42 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre> Responses <table><thead><tr><th>Code</th><th>Description</th><th>Links</th></tr></thead><tbody></tbody></table>	Code	Description	Links
Code	Description	Links		

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure : Researcher search functionality for locating researchers based on their names.

scholar-controller

GET /scholar/search-researcher

Parameters

Cancel

Name	Description
name * required	
string (query)	Rahul

Execute

Clear

Responses

Curl

curl -X 'GET' \n 'http://localhost:8080/scholar/search-researcher?name=Rahul' \n -H 'accept: */*'

Request URL

http://localhost:8080/scholar/search-researcher?name=Rahul

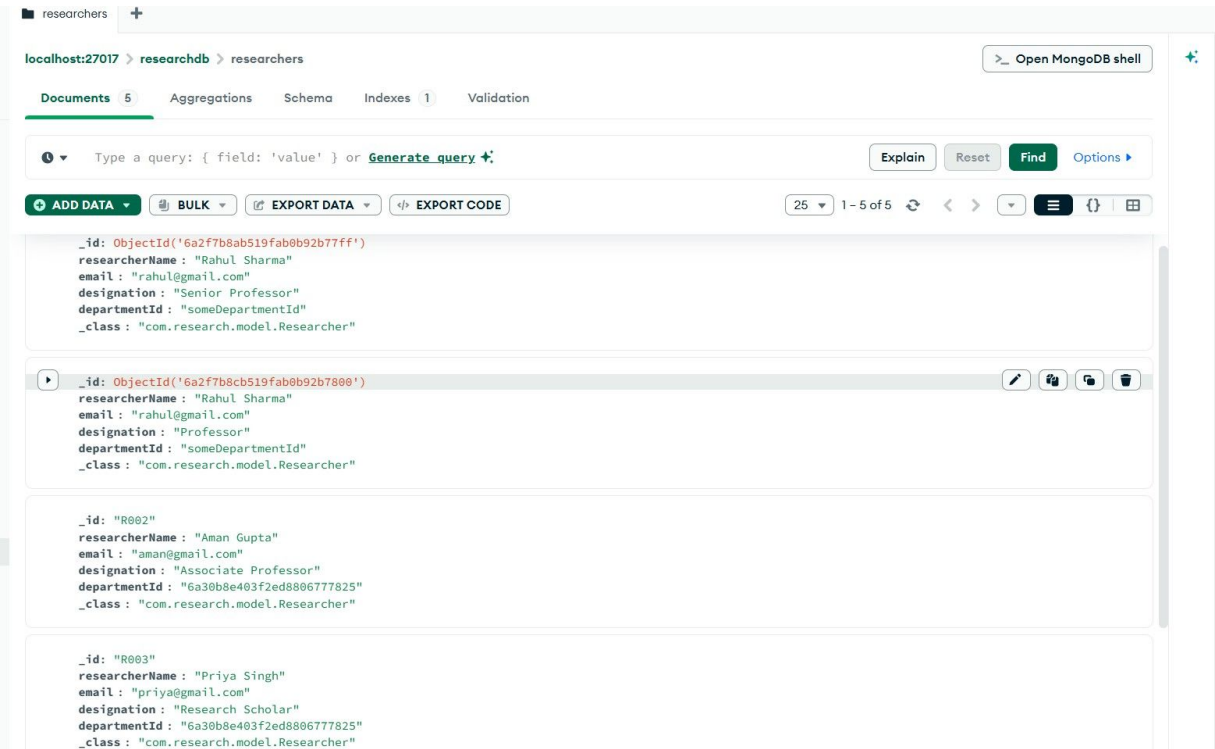
Server response

Code	Details
200	Response body <pre>{ "id": "6a2f78ab519fab8d2b77ff", "researcherName": "Rahul Sharma", "email": "rahul@gmail.com", "designation": "Senior Professor", "departmentId": "someDepartmentId" }, { "id": "6a2f78ab519fab8d2b7880", "researcherName": "Rahul Sharma", "email": "rahul@gmail.com", "designation": "Professor", "departmentId": "someDepartmentId" }, { "id": "6a2f882ee6425cf4157c9b12", "researcherName": "Rahul Sharma", "email": "rahul@gmail.com", "designation": "Professor", "departmentId": "CS001" }]</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Tue, 16 Jun 2026 22:51:46 GMT keep-alive: timeout=60 transfer-encoding: chunked</pre>

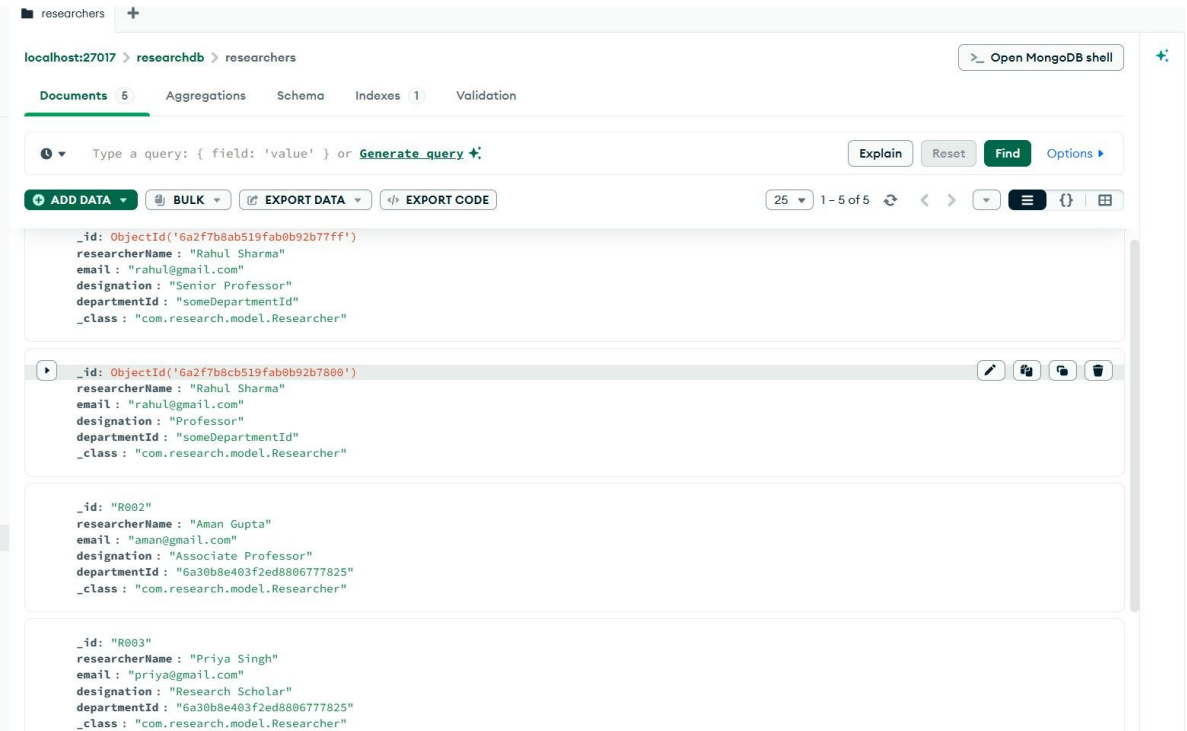
Responses

AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Figure : One-to-Many relationship between Department and Researchers.



Many-to-One relationship between Researchers and Department, where multiple researchers are associated with a single department through the departmentId reference.



AI ASSISTED RESEARCH PUBLICATION MANAGEMENT
SYSTEM CREATED BY: SAHIL KUMAR SONI 24BCE10085

Fig : Conference records stored in MongoDB for managing conference information and publication events.

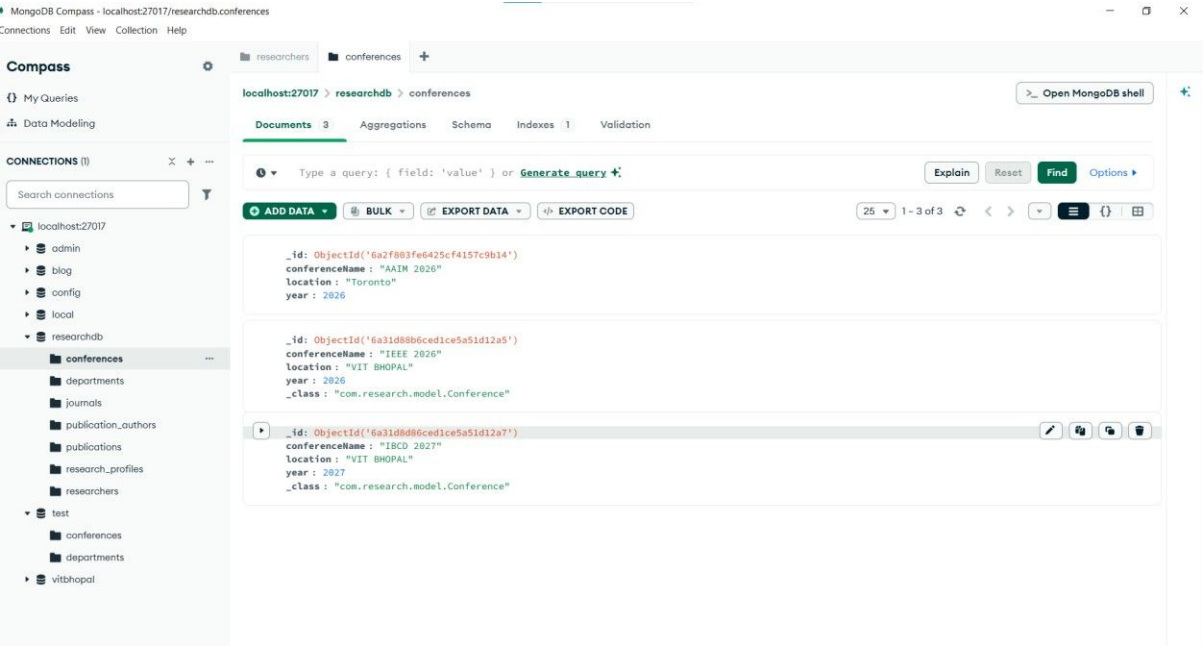


Fig: Journal records stored in MongoDB for publication categorization and management.

